Heart Disease Risk Prediction : MLOps Project Report

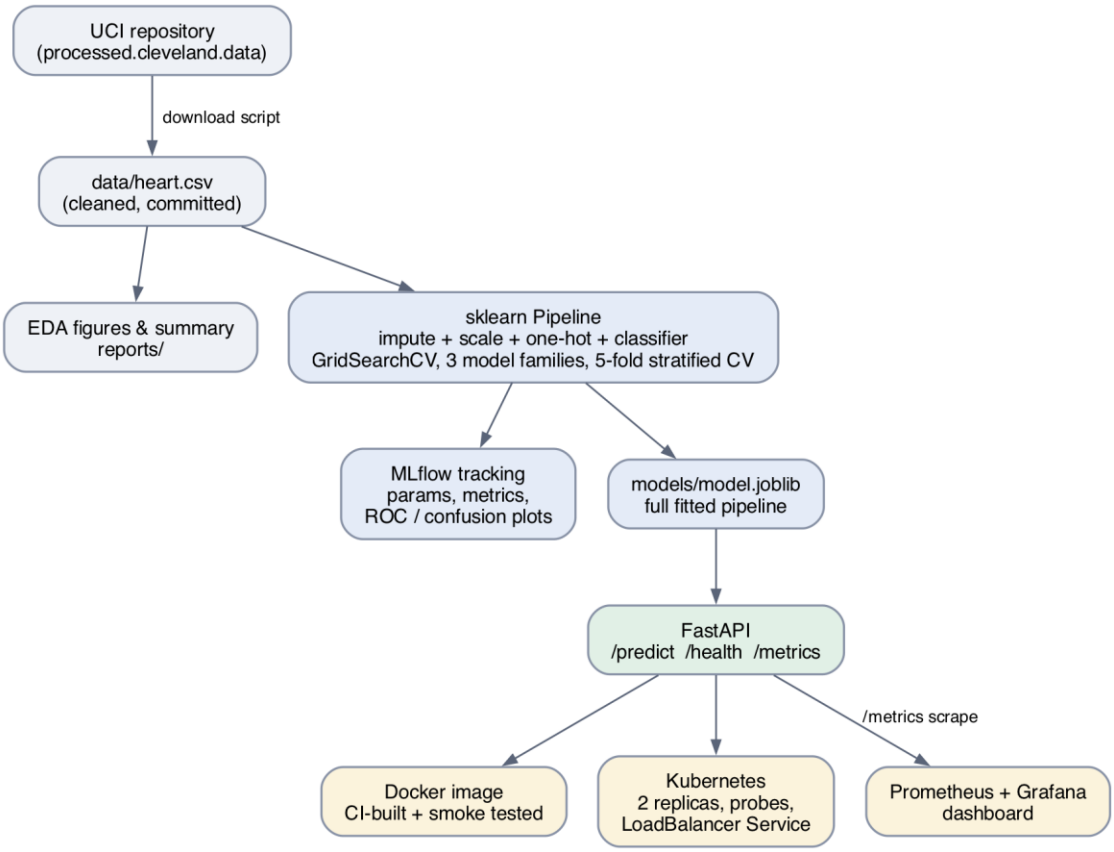
Table of Contents

Author: Diya Roy **Bits ID:** 2024AC05018 **Repository:** https://github.com/diyaRoy46/heart-disease-mlops

1. Project overview

This project builds and productionizes a binary classifier that predicts the risk of heart disease from routine patient measurements, using the UCI Heart Disease dataset (Cleveland subset, 303 patients, 13 features). The emphasis is on the *system around the model*: reproducible data acquisition, tracked experiments, automated quality gates, containerized serving, declarative deployment and live monitoring.

Pipeline at a glance



Pipeline overview

2. Data acquisition & preparation

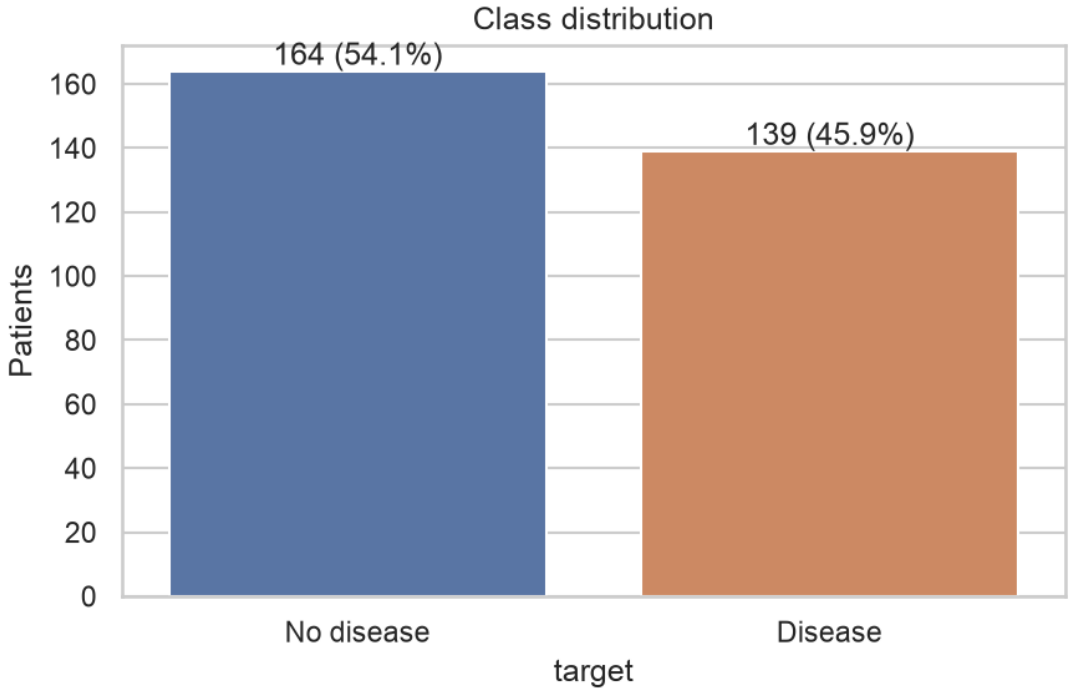
- python -m scripts.download_data fetches processed.cleveland.data from the UCI repository, names the 14 columns, parses ? as missing, binarizes the target (num > 0 → 1) and writes data/heart.csv (committed for reproducibility; the raw file is re-downloadable at any time).
- **Missing values:** only ca (4) and thal (2). They are *not* filled during cleaning — imputation happens inside the model pipeline so it is fitted on training folds only and replayed identically at inference time.
- **Encoding/scaling:** ColumnTransformer with median imputation + standard scaling for the 5 numeric features, and mode imputation + one-hot encoding (handle_unknown="ignore") for the 8 categorical features.

3. EDA findings

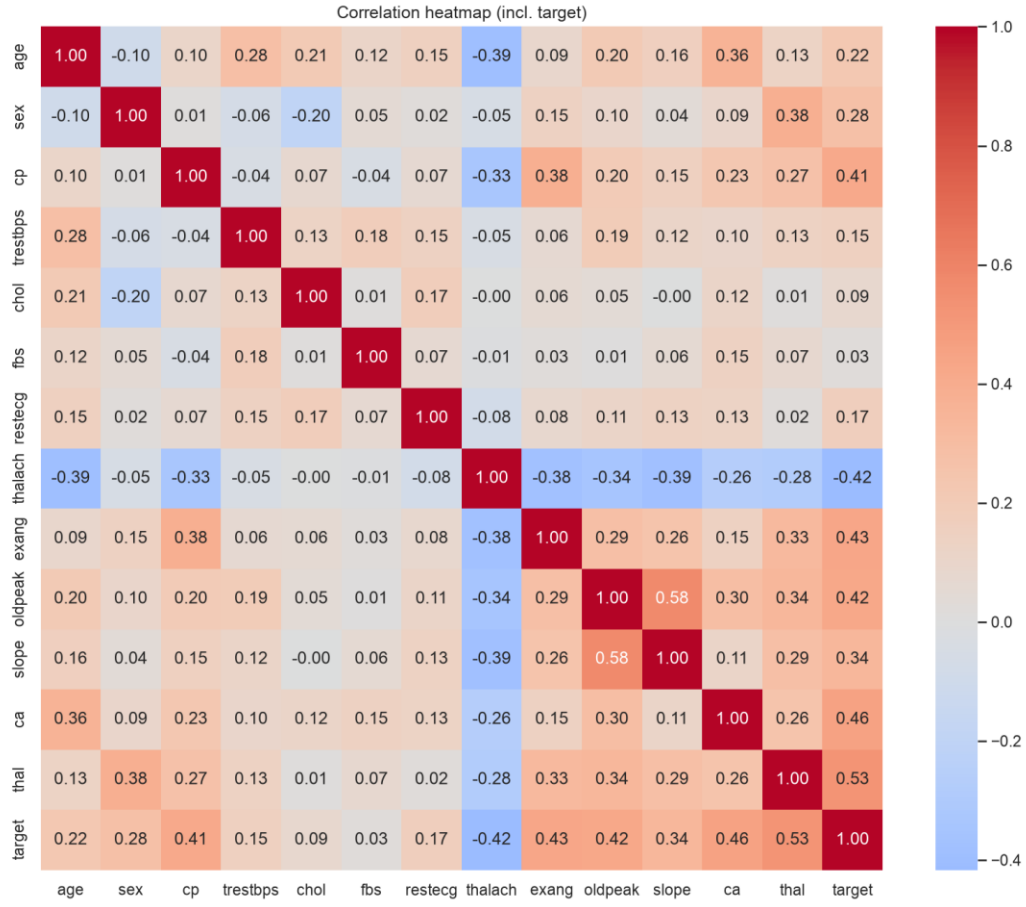
Figures in **reports/figures/**, summary in **reports/eda_summary.md**.

- **Class balance:** 164 without disease (54%) vs 139 with disease (46%) - near-balanced, so accuracy is meaningful but ROC-AUC is used for model selection anyway.

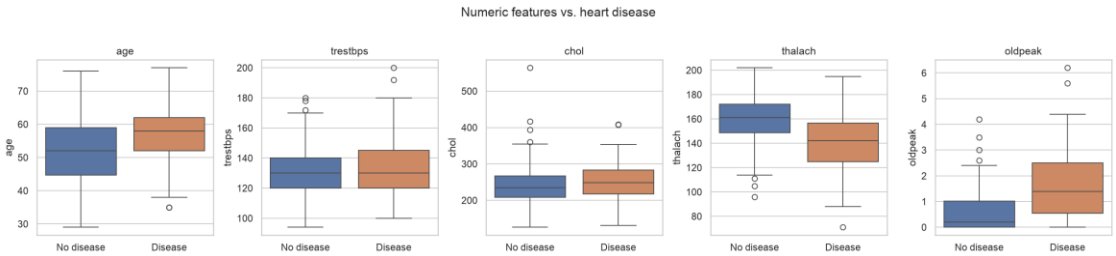
- **Strongest univariate signals** (|correlation| with target): thal (0.53), ca (0.46), exang (0.43), oldpeak (0.43), thalach (−0.42), cp (0.41). Clinically plausible: exercise-related measurements dominate.
- Patients with disease show **lower maximum heart rate** (thalach) and **higher exercise-induced ST depression** (oldpeak); asymptomatic chest pain (cp = 4) is heavily disease-associated.
- chol and fbs correlate weakly with the target (0.09 / 0.03) but are kept: tree models can still exploit interactions, and the cost is negligible.
- No degenerate columns; one exact duplicate policy applied (dedup in cleaning).



Class distribution



Correlation heatmap



Numeric features vs. heart disease

Further figures in **reports/figures/: 02_missing_values.png, 03_histograms.png, 06_categorical_by_target.png**.

4. Model development & comparison

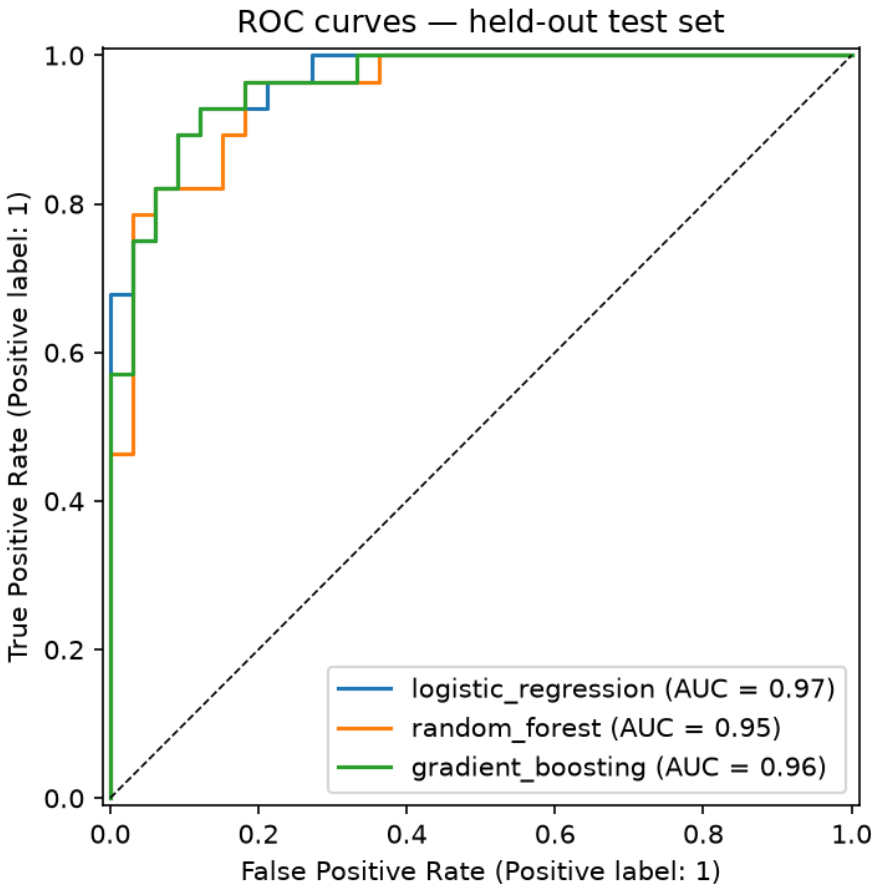
Three model families were tuned with GridSearchCV (5-fold stratified CV, ROC-AUC objective) on an 80/20 stratified split (seed 42), each wrapped in the same preprocessing pipeline:

Model	Grid	Best params
Logistic Regression	C ∈ {0.01, 0.1, 1, 10} × class_weight ∈ {None, balanced}	C=0.1, balanced
Random Forest	trees {200, 400} × depth {4, 8, ∅} × min_leaf {1, 3}	200 trees, depth 4, leaf 3
Gradient Boosting	trees {100, 200} × lr {0.05, 0.1} × depth {2, 3}	(see MLflow run)

Held-out test results (61 patients):

Model	Accuracy	Precision	Recall	F1	ROC-AUC	CV ROC-AUC
Logistic Regression	0.885	0.839	**0.929**	0.881	**0.965**	0.903 ± 0.017
Random Forest	0.852	0.806	0.893	0.847	0.951	0.902 ± 0.030
Gradient Boosting	**0.902**	**0.867**	**0.929**	**0.897**	0.961	0.874 ± 0.025

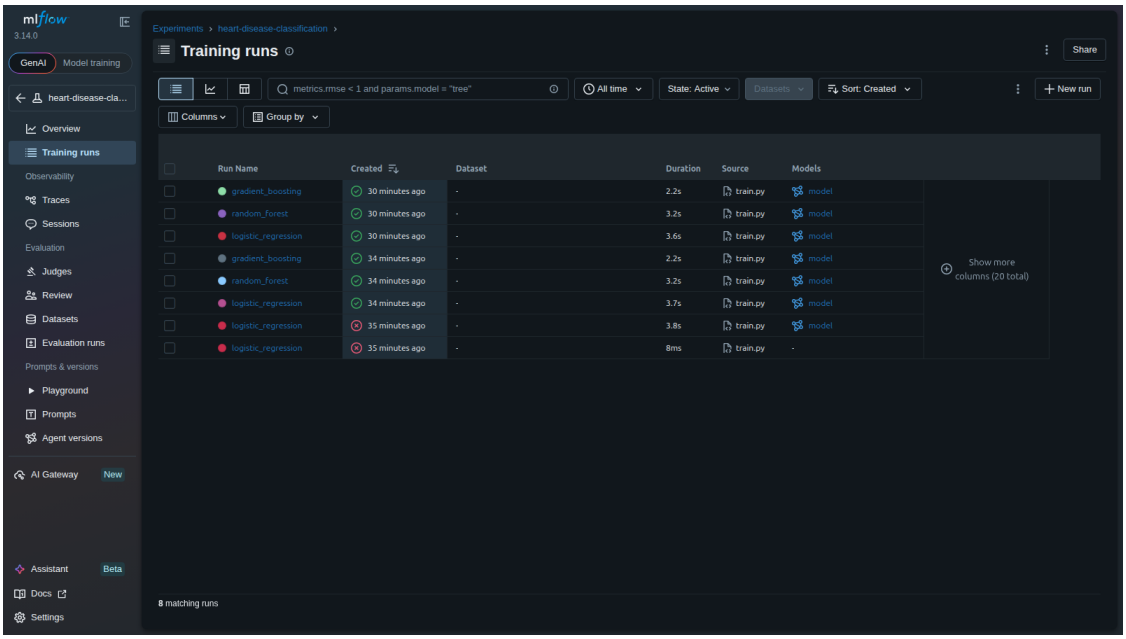
Selection: Logistic Regression was exported (best test ROC-AUC, best CV stability, and the simplest/most interpretable model - a sensible property for a clinical screening aid). With ~300 rows, heavily regularized linear models are hard to beat. Recall of 0.93 means few missed disease cases; the class_weight=balanced setting deliberately trades a little precision for recall, which is the right direction for screening.



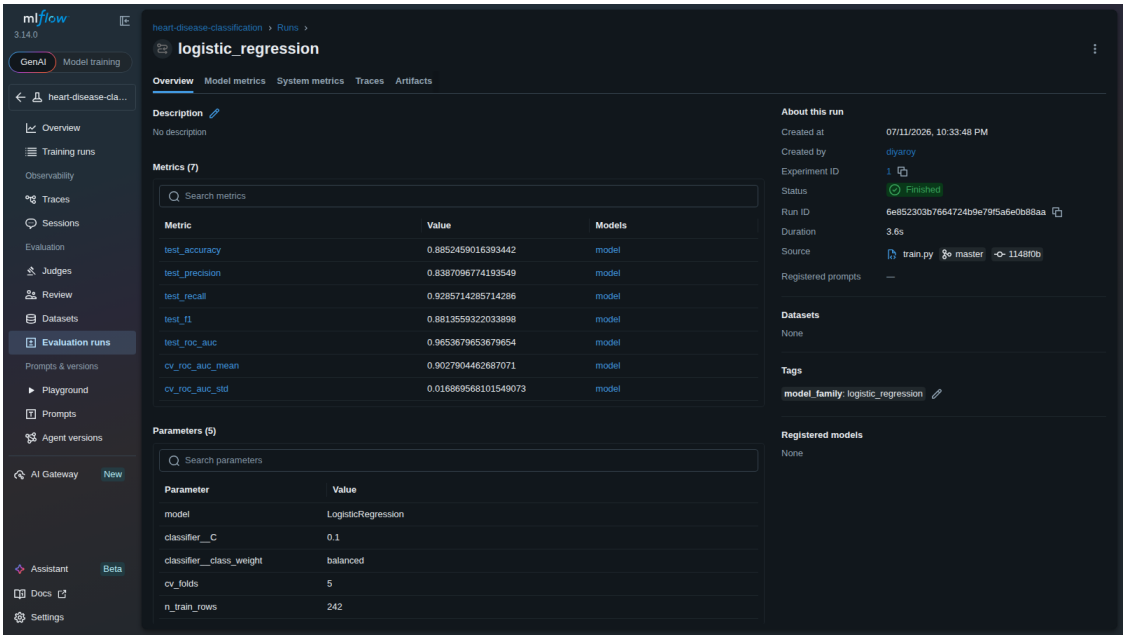
ROC curves — held-out test set

5. Experiment tracking (MLflow)

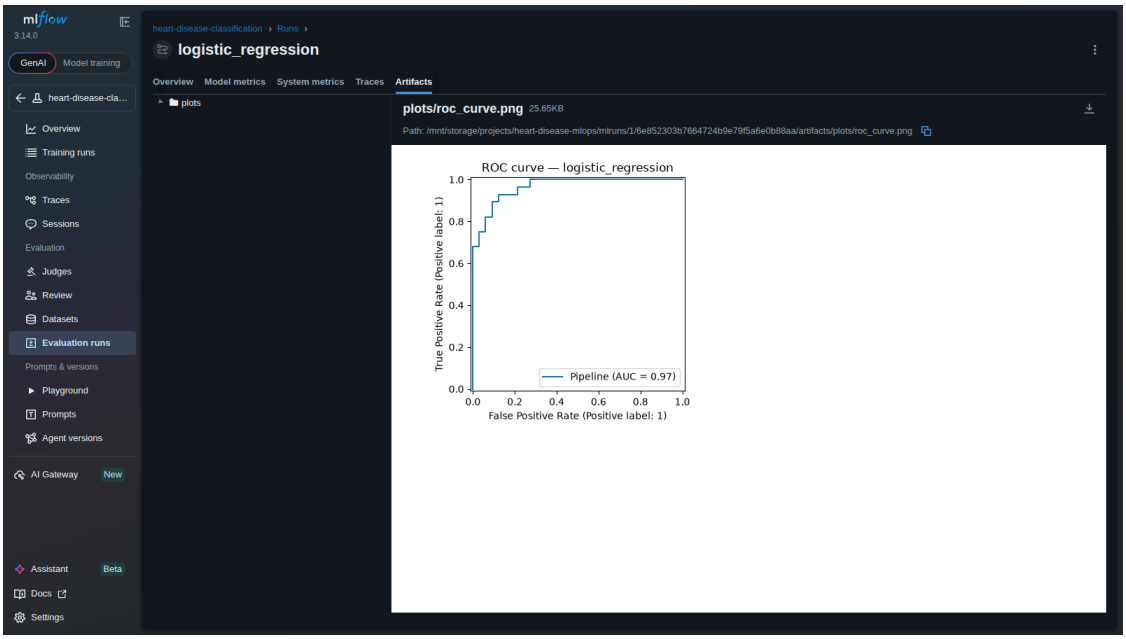
- Tracking store: sqlite:///mlflow.db (MLflow 3.x deprecates the ./mlruns file store), override with MLFLOW_TRACKING_URI.
- Each model family = one run under experiment heart-disease-classification, logging: best hyperparameters, CV mean/std, all five test metrics, per-run **confusion matrix** and **ROC curve** PNGs, and the fitted pipeline as an MLflow sklearn model with input example.
- Browse with mlflow ui --backend-store-uri sqlite:///mlflow.db.



MLflow — training runs



MLflow - run overview: parameters and metrics



MLflow - logged artifacts (ROC curve)

6. Model packaging & reproducibility

- Export format: single joblib file containing the **entire pipeline** (imputers, scaler, encoder, classifier) plus **models/metadata.json** (model name, params, metrics, MLflow run id, library versions, timestamp).
- requirements.txt (training) and requirements-serve.txt (serving, with model-critical packages pinned exactly) recreate both environments cleanly.
- Seeds fixed in **src/config.py**; retraining from the committed CSV reproduces the comparison table.

7. Serving API (FastAPI)

- **POST /predict**: validated JSON in (pydantic ranges from the UCI codebook; ca/thal optional), prediction + probability + model version out. Out-of-range input → HTTP 422; model failure → HTTP 500 with logged traceback.
- **GET /health:** used by Docker HEALTHCHECK and k8s probes.
- **GET /metrics:** Prometheus exposition (request counts/latency histograms via prometheus-fastapi-instrumentator, plus custom model_predictions_total counter labelled by predicted class).
- Every request is logged: method, path, status, latency, client.
- Interactive docs at /docs (Swagger UI).

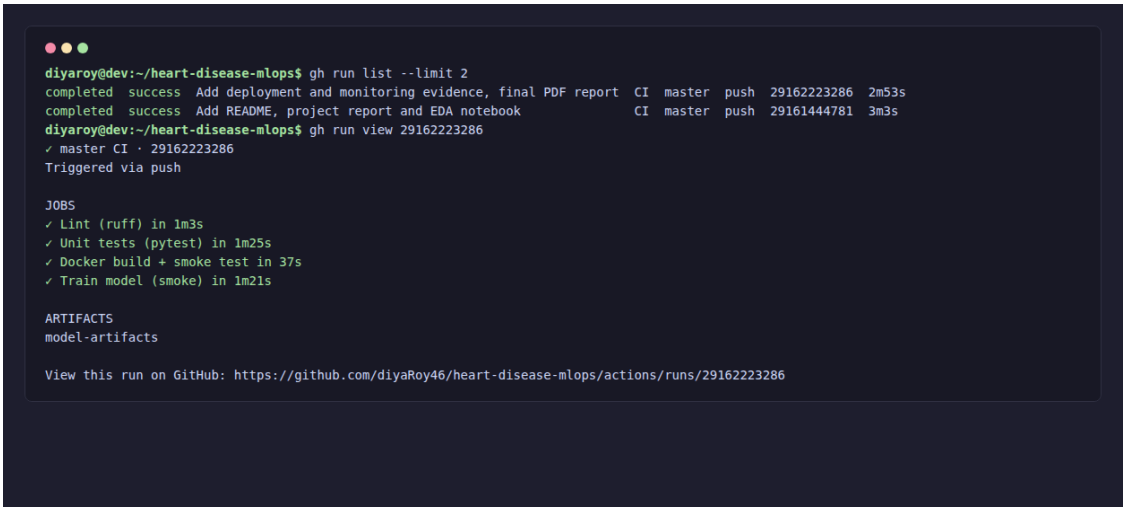
8. Testing & CI/CD

Tests (pytest, 16 cases): data cleaning (target binarization, parsing, dedup, schema of the committed CSV), preprocessing (no NaN survives, unseen categories don't crash), training (metrics bounded, quick end-to-end run logs to a temporary MLflow store, ROC-AUC > 0.8), API (health, valid/invalid predict, optional fields, metrics endpoint).

GitHub Actions (.github/workflows/ci.yml):

Job	Gate
lint	ruff — fails the pipeline on style/bug-pattern violations
test	pytest + coverage — fails on any test failure
train	reduced-grid training; uploads model, comparison and MLflow DB as artifacts
docker	builds the image, boots the container, polls /health, smoke-tests /predict

train and docker only run after lint and test succeed, so a red check blocks the artifact and image stages — the pipeline stops and reports clearly.



CI pipeline run - all four jobs green

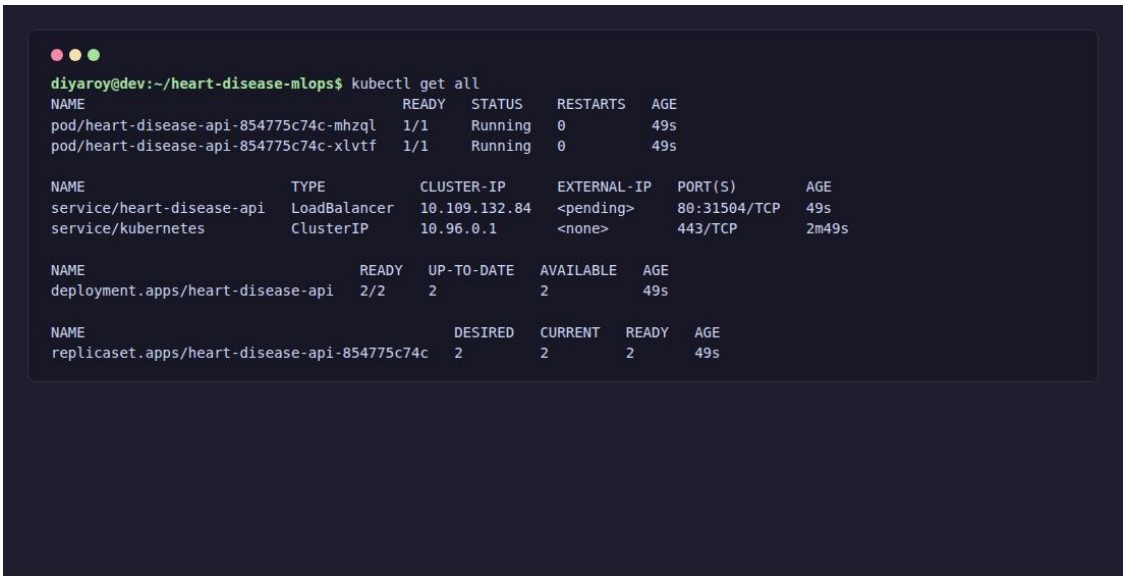
9. Deployment

Docker (verified locally): docker build -t heart-disease-api . → docker run -p 8000:8000 heart-disease-api → /health and /predict return correct responses. Image: python:3.14-slim, non-root user, HEALTHCHECK, serving-only dependencies.

Kubernetes (verified on Minikube v1.35 / Kubernetes v1.35.1): Deployment (2 replicas, readiness/liveness probes on /health, CPU/memory requests+limits, Prometheus scrape annotations) and a LoadBalancer Service (port 80 → 8000):

```
minikube image load heart-disease-api:latest   # or build via minikube docker-env
kubectl apply -f k8s/
minikube service heart-disease-api --url       # or: minikube tunnel
```

Both replicas reached Running/Ready and the API served predictions through the cluster service:



kubectl get all - pods, service, deployment

Verified prediction through the cluster service

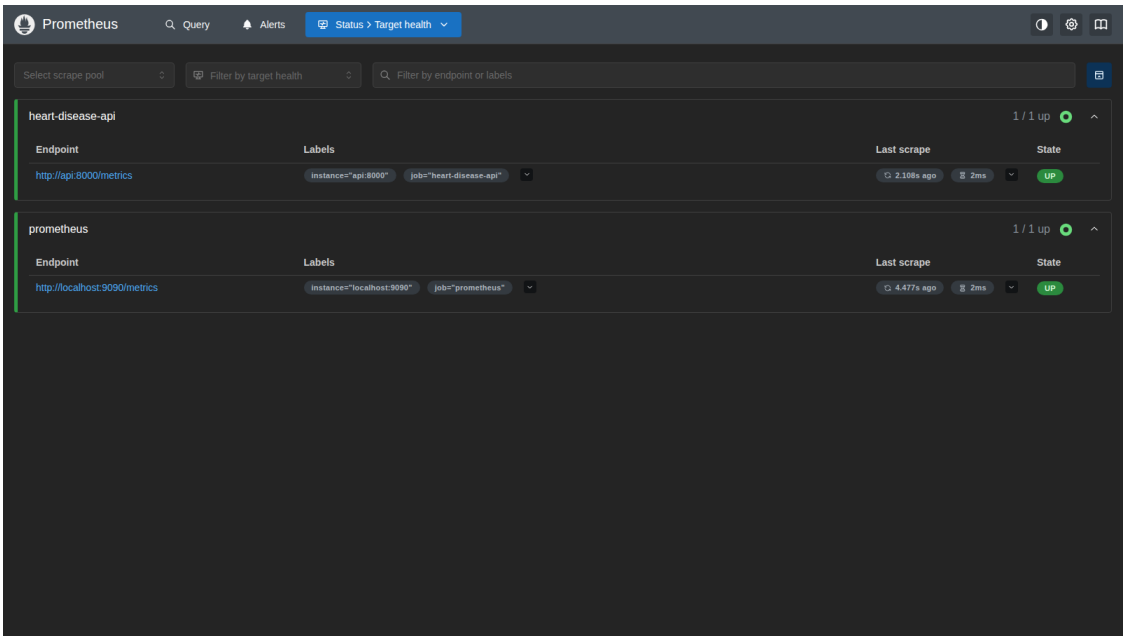
10. Monitoring & logging

docker compose up -d --build starts API + Prometheus + Grafana:

- Prometheus scrapes /metrics every 5 s (target health verified up).
- Grafana auto-provisions the **Heart Disease API** dashboard: request rate by endpoint, p50/p95 /predict latency, predictions per outcome class, and 4xx/5xx error rates.

- The prediction-outcome panel doubles as a crude **drift signal**: a sustained shift in the predicted-positive rate flags a change in incoming data.
- Structured request logs (docker logs heart-disease-api) cover every call.

Grafana - Heart Disease API dashboard



Prometheus - scrape targets up

11. Setup instructions (clean machine)

```
git clone https://github.com/diyaRoy46/heart-disease-mlops && cd heart-disease-mlops
python -m venv .venv && source .venv/bin/activate
pip install -r requirements-dev.txt
python -m scripts.download_data   # data
python -m src.eda                 # EDA figures
python -m src.train               # tracked training + model export
pytest && ruff check src tests scripts
uvicorn src.api.main:app --port 8000
# or the full stack:
docker compose up -d --build
```

12. Operational lessons & future work

A real training/serving-skew incident (and what caught it)

During final verification the containerized API began returning HTTP 500 on every /predict call while /health stayed green. The container logs showed:

```
AttributeError: 'SimpleImputer' object has no attribute '_fill_dtype'
```

Root cause: the model had been retrained on a host environment running scikit-learn **1.7.2**, while the serving image installs the exact pin scikit-learn==1.9.0 from requirements-serve.txt. The pickled imputer from 1.7.2 lacks an internal attribute that 1.9.0's transform() expects — a textbook train/serve skew. The fix was to align the training environment to the serving pins, retrain, and rebuild the image; models/metadata.json records sklearn_version per export, which is how the mismatch was confirmed in minutes rather than hours.

Two design decisions proved their worth here: **exact pins for model-critical packages in the serving image** (the drift surfaced immediately instead of corrupting predictions silently) and **version stamping in the model metadata** (made the diagnosis trivial). A concrete follow-up is listed below.

Deployment re-verification on kind

Beyond the Minikube verification in section 9, the same manifests were re-deployed unchanged onto a kind cluster (as shown in the demo video): kind load docker-image heart-disease-api:latest, kubectl apply -f k8s/, both replicas Ready, and predictions served through the Service via kubectl port-forward (kind provisions no external LoadBalancer IP). Running the identical YAML on two local distributions is a useful portability check before any cloud target.

Experiment tracking summary

The MLflow store accumulated multiple generations of runs over the project: full-grid runs (three model families), reduced-grid smoke runs (CI and demo), and the post-incident retrain. Every generation logged the same contract - params, CV mean/std, five test metrics, ROC/confusion artifacts, fitted pipeline, so results stayed comparable across library versions, and the final exported model is traceable to run 6ffa9b28… in metadata.json.

Future work

- **Fail-fast version guard:** assert at API startup that the running scikit-learn version equals metadata.json's sklearn_version, refusing to serve on mismatch instead of failing at first prediction.
- **Continuous delivery:** push the CI-built image to a registry and roll the Kubernetes Deployment automatically on green pipelines.
- **Drift detection:** extend the predicted-positive-rate panel with input feature distribution monitoring (e.g. evidently) and alerting rules in Prometheus.
- **Model registry:** promote exports through MLflow Model Registry stages (Staging → Production) rather than a single committed artifact.